

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Tomás San Martín

11 de septiembre de 2026

17:41

1. Introducción

La notación asintótica acota el crecimiento de un algoritmo, pero no informa sobre constantes ocultas, jerarquía de memoria ni el efecto del tipo de entrada. Este trabajo estudia experimentalmente cuatro algoritmos de ordenamiento y dos de multiplicación de matrices cuadradas, midiendo tiempo y memoria sobre 72 casos por problema (Anexo A), y contrasta lo observado con las cotas teóricas $O(n \log n)$, $O(n^3)$ y $O(n^{\log_2 7})$. El código, los datos y los scripts de medición y graficado se incluyen en el directorio `code/`.

2. Implementaciones

El código de la tarea se encuentra subido en el repositorio de GitHub adjuntado:

<https://github.com/TomySNM/inf221-tarea-1>

3. Entorno experimental

Equipo: Intel(R) Core(TM) i5-9300H CPU @ 2.40GHz, 24 GB RAM, Windows 11 con WSL2 (Ubuntu); compilador g++ 13.3.0 con flags `-O2 -std=c++17` (en todos los algoritmos).

Se empleó el nivel de optimización `-O2` por tres razones. En primera instancia, porque nuestro objetivo es medir y estudiar el comportamiento de los algoritmos en la práctica. En segundo lugar, por factibilidad experimental, ya que, de lo contrario, en las pruebas de Strassen, el tiempo de

ejecución aumentaría considerablemente. Y en tercer lugar, al medir en un entorno igual para todos (mismas flags), las diferencias de las curvas las marcan los algoritmos y no el compilador.

Los casos de prueba se generaron con los scripts provistos por el curso: ordenamiento con $N = \{10, 10^3, 10^5, 10^7\}$, $T = \{\text{ascendente, descendente, aleatorio}\}$, $D = \{D1, D7\}$, $M = \{a, b, c\}$; matrices con $N = \{2^4, 2^6, 2^8, 2^{10}\}$, $T = \{\text{dispersa, diagonal, densa}\}$, $D = \{D0, D10\}$, $M = \{a, b, c\}$ (72 casos por problema). Como limitaciones en el desarrollo de la tarea tenemos efectos de caché al momento de comparar teórica y experimentalmente los algoritmos, y la resolución del muestreo de memoria.

4. Resultados y Análisis

4.1. Ordenamiento

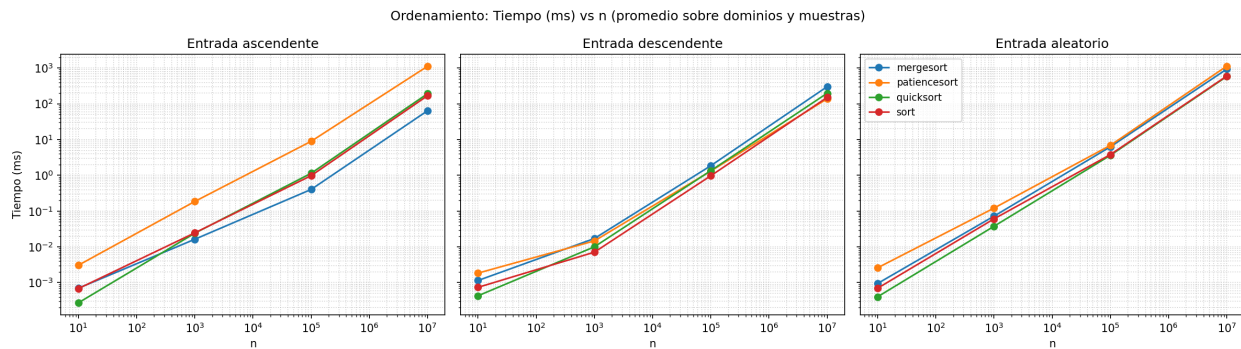


Figura 1: Tiempo de ordenamiento vs n (escala log-log), promedio sobre dominios y muestras.

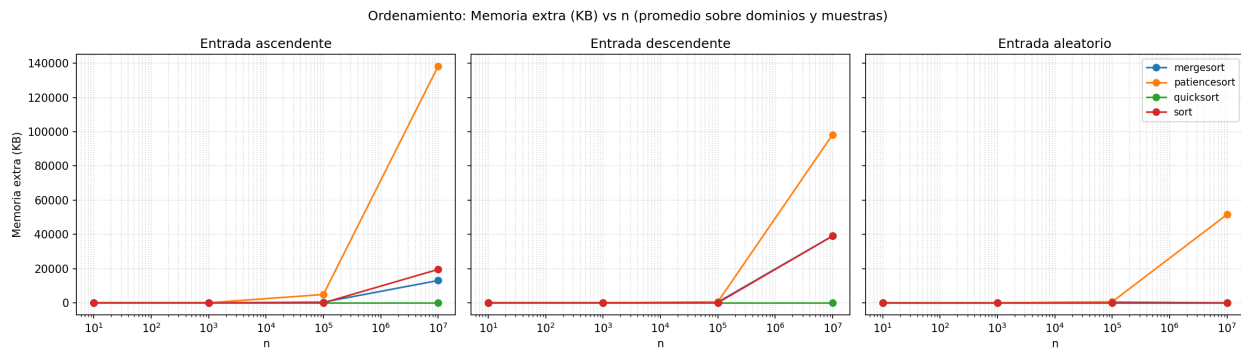


Figura 2: Memoria extra de ordenamiento vs n .

Cuadro 1: Tiempo (ms), entrada aleatoria, dominio D7, muestra a.

n	merge	quick	patience	sort
10^5	7.98	5.17	10.26	5.42
10^7	1247.82	867.64	1870.45	862.97

En escala log-log las curvas presentan pendiente cercana a 1, coherente con $O(n \log n)$. sort y quick sort dominan en entrada aleatoria; merge sort es el más rápido en entrada ascendente gracias a la fusión condicional, mientras que patience sort se degrada notablemente en ese caso porque genera aproximadamente n pilas. En memoria, merge y patience crecen lineales ($O(n)$: buffer y pilas más arreglo de salida), en tanto que quick sort y sort permanecen planos: su espacio extra es pila de recursión $O(\log n)$, no heap.

4.2. Multiplicación de matrices

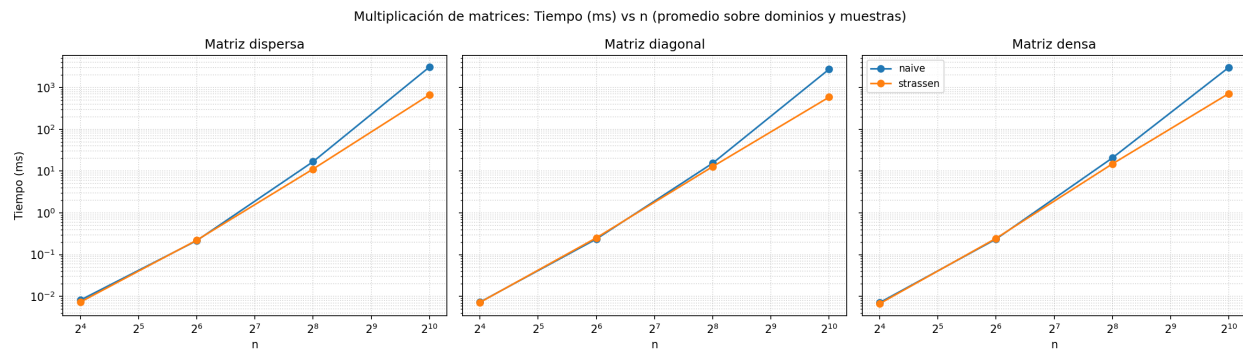


Figura 3: Tiempo de multiplicación vs n (log-log).

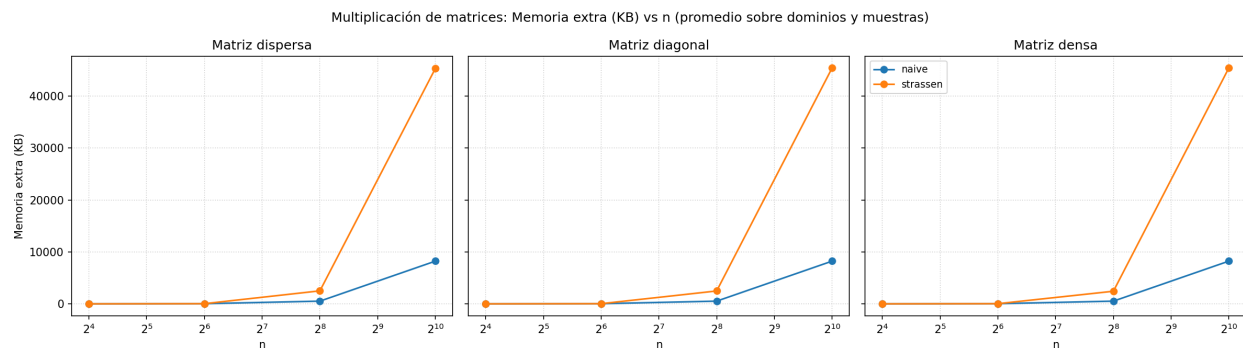


Figura 4: Memoria extra de multiplicación vs n .

Cuadro 2: Tiempo (ms), matriz densa, dominio D10, muestra a.

n	naive	strassen
2^8	16.62	13.24
2^{10}	3296.23	712.13

Naive exhibe pendiente ≈ 3 ($O(n^3)$) y Strassen $\approx 2,81$ ($O(n^{\log_2 7})$). El cruce aparece entre 2^8 y 2^{10} : la constante oculta de Strassen (siete productos recursivos más las sumas de matrices) solo se amortiza en tamaños grandes. En memoria, naive asigna únicamente la matriz resultado (≈ 8 MB en

$n = 1024$), mientras Strassen alcanza ≈ 45 MB por los temporales recursivos: el divide-and-conquer compra tiempo con espacio.

4.3. Contraste con el teorema maestro

Para cerrar el círculo entre teoría y práctica usamos el teorema maestro como una predicción: a partir de cómo cada algoritmo divide su problema, el teorema nos dice qué crecimiento deberíamos ver en los gráficos. Merge sort, por ejemplo, parte el arreglo en dos mitades y luego dedica un trabajo proporcional a n para volver a juntarlas; esa es justamente la recurrencia $T(n) = 2T(n/2) + \Theta(n)$, y el teorema (caso 2) responde que el tiempo debe crecer como $n \log n$; el quick sort promedio, con partición balanceada, tiene la misma recurrencia y por tanto la misma cota. Strassen hace algo distinto: resuelve 7 multiplicaciones con la mitad del tamaño y deja para el final un trabajo de n^2 en sumas y restas de bloques; como ese trabajo final crece más lento que $n^{\log_2 7}$, el teorema (caso 1) predice un crecimiento de $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$. Y si pensamos la multiplicación naive con la misma lógica (8 mitades más un trabajo de n^2), el teorema devuelve el conocido $\Theta(n^3)$.

Lo valioso es que esas predicciones se cumplen en los datos. Entre $n = 10^5$ y $n = 10^7$ (Cuadro 1) los tiempos se multiplican entre 156 y 182 veces, lo que en la pendiente del gráfico log-log equivale a un exponente de 1.10 a 1.13: en palabras simples, el $n \log n$ que el teorema había anunciado. En matrices, Strassen multiplica su tiempo por 53.8 al pasar de 2^8 a 2^{10} (Cuadro 2), es decir, un exponente medido de 2.87 contra el 2.81 teórico. El único que se sale del libreto es naive, con un exponente medido de 3.8 frente al 3 esperado; pero más que contradecir la teoría, ese desvío la complementa: a ese tamaño las tres matrices de 8 MB ya no caben en la caché del procesador, y buena parte del tiempo se va esperando datos desde la RAM, justamente la limitación que declaramos en la Sección 3. En síntesis: cuando el tamaño es suficiente para que mande el comportamiento asintótico, la teoría y el cronómetro cuentan la misma historia; y cuando no coinciden, la diferencia se explica por la jerarquía de memoria.

5. Conclusiones

El comportamiento práctico reproduce las cotas teóricas en el régimen asintótico, pero la constante oculta, el tipo de entrada y las decisiones de implementación (mediana de tres, fusión con min-heap, umbral de corte en Strassen) determinan qué algoritmo conviene en cada rango de n . El análisis experimental es, por tanto, el complemento necesario de la notación asintótica para seleccionar algoritmos en la práctica, ya que también entran en juego la jerarquía de memoria y la eficiencia de caché, que no se reflejan en la complejidad teórica, y que afectan en lo que podríamos llegar a esperar.

Referencias

- [1] cppreference. *std::sort*. <https://en.cppreference.com/w/cpp/algorithm/sort>. 2026.

- [2] GeeksforGeeks. *Patience Sorting*. <https://www.geeksforgeeks.org/dsa/patience-sorting/>. Base de la fase de pilas; la fusión fue reescrita con un min-heap por el autor. 2026.
- [3] Robert Sedgewick. «Implementing Quicksort Programs». En: *Communications of the ACM* 21.10 (1978), págs. 847-857.
- [4] Volker Strassen. «Gaussian elimination is not optimal». En: *Numerische Mathematik* 13 (1969), págs. 354-356.
- [5] Wikipedia. *Patience sorting*. https://en.wikipedia.org/wiki/Patience_sorting. 2026.
- [6] Wikipedia. *Strassen algorithm*. https://en.wikipedia.org/wiki/Strassen_algorithm. 2026.